



Lab00 - Protocol SCTP

TSM - Advanced Communication Architecture

Group AR-2 : Dimitri Julmy, Dylan Flotte

-

University of Applied Sciences and Arts of Western Switzerland (HES-SO)
Master of Science in Engineering (MSE)
Information and cyber security (ICS)

Repository URI	https://github.com/HezelTm/tsm_advcomarc
Creation date	2026-03-18
Submission date	2026-03-26

Introduction

TODO

Implementation

Analyse de la capture Wireshark

(a) Ouvrir le fichier SCTP-ADDi.pcap dans Wireshark

La capture sctp-addip.pcap contient une communication SCTP de 38 paquets entre trois adresses IP différentes (192.168.0.100, 192.168.0.101 et 192.168.0.102).

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	192.168.0.101	192.168.0.100	SCTP	84	INIT
2	0.000292	192.168.0.100	192.168.0.101	SCTP	240	INIT_ACK
3	0.000355	192.168.0.101	192.168.0.100	SCTP	212	COOKIE_ECHO
4	0.000568	192.168.0.100	192.168.0.101	SCTP	62	COOKIE_ACK
5	0.000769	192.168.0.101	192.168.0.100	SCTP	92	DATA (TSN=0)
6	0.000890	192.168.0.101	192.168.0.100	SCTP	80	ASCONF
7	0.000920	192.168.0.100	192.168.0.101	SCTP	64	SACK (Ack=0, Arwnd=109542)
8	0.001027	192.168.0.100	192.168.0.101	SCTP	62	ASCONF_ACK
9	0.001939	192.168.0.100	192.168.0.101	SCTP	1064	DATA (TSN=0)
10	0.002052	192.168.0.101	192.168.0.100	SCTP	64	SACK (Ack=0, Arwnd=108568)
11	0.007854	192.168.0.101	192.168.0.100	SCTP	92	DATA (TSN=1)
12	0.008356	192.168.0.100	192.168.0.101	SCTP	1080	SACK (Ack=1, Arwnd=109516) DATA (TSN=1)
13	0.008457	192.168.0.100	192.168.0.101	SCTP	1064	DATA (TSN=2)
14	0.008571	192.168.0.101	192.168.0.100	SCTP	64	SACK (Ack=2, Arwnd=107568)
15	0.008557	192.168.0.100	192.168.0.101	SCTP	1064	DATA (TSN=3)
16	0.010567	192.168.0.101	192.168.0.100	SCTP	92	DATA (TSN=2)
17	0.011184	192.168.0.101	192.168.0.100	SCTP	80	ASCONF
18	0.011353	192.168.0.100	192.168.0.102	SCTP	62	ASCONF_ACK
19	0.012130	192.168.0.101	192.168.0.100	SCTP	92	DATA (TSN=3)
20	0.012320	192.168.0.100	192.168.0.101	SCTP	64	SACK (Ack=3, Arwnd=109516)
21	0.013416	192.168.0.101	192.168.0.100	SCTP	64	SACK (Ack=3, Arwnd=109568)
22	0.014040	192.168.0.101	192.168.0.100	SCTP	92	DATA (TSN=4)
23	0.030749	192.168.0.100	192.168.0.102	SCTP	1064	DATA (TSN=4)
24	0.043514	192.168.0.100	192.168.0.102	SCTP	1064	DATA (TSN=5)
25	0.043654	192.168.0.101	192.168.0.100	SCTP	64	SACK (Ack=5, Arwnd=107568)
26	0.056957	192.168.0.100	192.168.0.102	SCTP	1064	DATA (TSN=6)
27	0.107983	192.168.0.102	192.168.0.100	SCTP	80	ASCONF
28	0.108188	192.168.0.100	192.168.0.102	SCTP	62	ASCONF_ACK
29	0.108299	192.168.0.102	192.168.0.100	SCTP	92	DATA (TSN=5)
30	0.108445	192.168.0.100	192.168.0.102	SCTP	64	SACK (Ack=5, Arwnd=109516)
31	0.108464	192.168.0.102	192.168.0.100	SCTP	92	DATA (TSN=6)
32	0.109175	192.168.0.102	192.168.0.100	SCTP	64	SACK (Ack=6, Arwnd=109568)
33	0.109252	192.168.0.102	192.168.0.100	SCTP	92	DATA (TSN=7)
34	0.109345	192.168.0.100	192.168.0.102	SCTP	62	SHUTDOWN
35	0.109419	192.168.0.100	192.168.0.102	SCTP	64	SACK (Ack=7, Arwnd=109516)
36	0.109508	192.168.0.102	192.168.0.100	SCTP	52	SHUTDOWN_ACK
37	0.109449	192.168.0.100	192.168.0.102	SCTP	62	SHUTDOWN
38	0.109634	192.168.0.100	192.168.0.102	SCTP	62	SHUTDOWN_COMPLETE

(b) Appliquer un filtre pour afficher uniquement les paquets SCTP : sctp

L'application du filtre sctp ne change rien par rapport à l'affichage initial, car tous les paquets de la capture sont des paquets SCTP.

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	192.168.0.101	192.168.0.100	SCTP	84	INIT
3	0.000355	192.168.0.101	192.168.0.100	SCTP	212	COOKIE_ECHO
5	0.000769	192.168.0.101	192.168.0.100	SCTP	92	DATA (TSN=0)
6	0.000890	192.168.0.101	192.168.0.100	SCTP	80	ASCONF
10	0.002052	192.168.0.101	192.168.0.100	SCTP	64	SACK (Ack=0, Arwnd=108568)
11	0.007854	192.168.0.101	192.168.0.100	SCTP	92	DATA (TSN=1)
14	0.008571	192.168.0.101	192.168.0.100	SCTP	64	SACK (Ack=2, Arwnd=107568)
16	0.010567	192.168.0.101	192.168.0.100	SCTP	92	DATA (TSN=2)
17	0.011184	192.168.0.101	192.168.0.100	SCTP	80	ASCONF
19	0.012130	192.168.0.101	192.168.0.100	SCTP	92	DATA (TSN=3)
21	0.013416	192.168.0.101	192.168.0.100	SCTP	64	SACK (Ack=3, Arwnd=109568)
22	0.014040	192.168.0.101	192.168.0.100	SCTP	92	DATA (TSN=4)
25	0.043654	192.168.0.101	192.168.0.100	SCTP	64	SACK (Ack=5, Arwnd=107568)
27	0.107983	192.168.0.102	192.168.0.100	SCTP	80	ASCONF
29	0.108299	192.168.0.102	192.168.0.100	SCTP	92	DATA (TSN=5)
31	0.108464	192.168.0.102	192.168.0.100	SCTP	92	DATA (TSN=6)
32	0.109175	192.168.0.102	192.168.0.100	SCTP	64	SACK (Ack=6, Arwnd=109568)
33	0.109252	192.168.0.102	192.168.0.100	SCTP	92	DATA (TSN=7)
36	0.109508	192.168.0.102	192.168.0.100	SCTP	52	SHUTDOWN_ACK
2	0.000292	192.168.0.100	192.168.0.101	SCTP	240	INIT_ACK
4	0.000568	192.168.0.100	192.168.0.101	SCTP	62	COOKIE_ACK
7	0.000920	192.168.0.100	192.168.0.101	SCTP	64	SACK (Ack=0, Arwnd=109542)
8	0.001027	192.168.0.100	192.168.0.101	SCTP	62	ASCONF_ACK
9	0.001939	192.168.0.100	192.168.0.101	SCTP	1064	DATA (TSN=0)
12	0.008356	192.168.0.100	192.168.0.101	SCTP	1080	SACK (Ack=1, Arwnd=109516) DATA (TSN=1)
13	0.008457	192.168.0.100	192.168.0.101	SCTP	1064	DATA (TSN=2)
15	0.008557	192.168.0.100	192.168.0.101	SCTP	1064	DATA (TSN=3)
20	0.012320	192.168.0.100	192.168.0.101	SCTP	64	SACK (Ack=3, Arwnd=109516)
18	0.011353	192.168.0.100	192.168.0.102	SCTP	62	ASCONF_ACK
23	0.030749	192.168.0.100	192.168.0.102	SCTP	1064	DATA (TSN=4)
24	0.043514	192.168.0.100	192.168.0.102	SCTP	1064	DATA (TSN=5)
26	0.056957	192.168.0.100	192.168.0.102	SCTP	1064	DATA (TSN=6)
28	0.108188	192.168.0.100	192.168.0.102	SCTP	62	ASCONF_ACK
30	0.108445	192.168.0.100	192.168.0.102	SCTP	64	SACK (Ack=5, Arwnd=109516)
34	0.109345	192.168.0.100	192.168.0.102	SCTP	62	SHUTDOWN
35	0.109419	192.168.0.100	192.168.0.102	SCTP	64	SACK (Ack=7, Arwnd=109516)
37	0.109449	192.168.0.100	192.168.0.102	SCTP	62	SHUTDOWN
38	0.109634	192.168.0.100	192.168.0.102	SCTP	62	SHUTDOWN_COMPLETE

(c) Identifier les adresses IP source et destination

Les adresses IP sources sont :

- 192.168.0.100
- 192.168.0.101
- 192.168.0.102

Les adresses IP destinations sont les mêmes.

(d) Repérer les différentes phases de la communication SCTP

La communication SCTP est divisée selon les phases suivantes :

1. SCTP association establishment message flow

L'établissement de l'association SCTP se fait en quatre étapes (*4-way handshake*) et implique les paquets 1 à 4, entre les stations 192.168.0.100 et 192.168.0.101.

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	192.168.0.101	192.168.0.100	SCTP	84	INIT
2	0.000292	192.168.0.100	192.168.0.101	SCTP	240	INIT_ACK
3	0.000355	192.168.0.101	192.168.0.100	SCTP	212	COOKIE_ECHO
4	0.000568	192.168.0.100	192.168.0.101	SCTP	62	COOKIE_ACK

La station A 192.168.0.100 initie l'association en envoyant à la station B 192.168.0.101 un paquet contenant un INIT chunk, qui peut inclure une ou plusieurs adresses IP utilisées par l'initiateur. La station

B 192.168.0.101 répond avec un paquet contenant un INIT_ACK chunk, qui peut également inclure une ou plusieurs adresses IP utilisées par le répondant. Les deux INIT chunk et INIT_ACK chunk spécifient le nombre de flux sortants supportés par l'association, ainsi que le nombre maximum de flux entrants acceptés de l'autre point de terminaison.

L'association est ensuite complétée par un échange de COOKIE ECHO/COOKIE ACK qui spécifie une valeur de cookie utilisée dans tous les échanges de données ultérieurs.

2. SCTP data transfer

Une fois l'association établie, les données peuvent être transférées entre les points de terminaison en utilisant des paquets DATA chunk. Le récepteur *acknowledges* la réception des données avec des paquets SACK chunk. Les paquets 5 à 35 (sauf le paquet 34) sont impliqués dans cette phase.

5	0.000769	192.168.0.101	192.168.0.100	SCTP	92 DATA (TSN=0)
6	0.000890	192.168.0.101	192.168.0.100	SCTP	80 ASCONF
7	0.000920	192.168.0.100	192.168.0.101	SCTP	64 SACK (Ack=0, Arwnd=109542)
8	0.001027	192.168.0.100	192.168.0.101	SCTP	62 ASCONF_ACK
9	0.001939	192.168.0.100	192.168.0.101	SCTP	1064 DATA (TSN=0)
10	0.002052	192.168.0.101	192.168.0.100	SCTP	64 SACK (Ack=0, Arwnd=108568)
11	0.007854	192.168.0.101	192.168.0.100	SCTP	92 DATA (TSN=1)
12	0.008356	192.168.0.100	192.168.0.101	SCTP	1080 SACK (Ack=1, Arwnd=109516) DATA (TSN=1)
13	0.008457	192.168.0.100	192.168.0.101	SCTP	1064 DATA (TSN=2)
14	0.008571	192.168.0.101	192.168.0.100	SCTP	64 SACK (Ack=2, Arwnd=107568)
15	0.008557	192.168.0.100	192.168.0.101	SCTP	1064 DATA (TSN=3)
16	0.010567	192.168.0.101	192.168.0.100	SCTP	92 DATA (TSN=2)
17	0.011184	192.168.0.101	192.168.0.100	SCTP	80 ASCONF
18	0.011353	192.168.0.100	192.168.0.102	SCTP	62 ASCONF_ACK
19	0.012130	192.168.0.101	192.168.0.100	SCTP	92 DATA (TSN=3)
20	0.012320	192.168.0.100	192.168.0.101	SCTP	64 SACK (Ack=3, Arwnd=109516)
21	0.013416	192.168.0.101	192.168.0.100	SCTP	64 SACK (Ack=3, Arwnd=109568)
22	0.014040	192.168.0.101	192.168.0.100	SCTP	92 DATA (TSN=4)
23	0.030749	192.168.0.100	192.168.0.102	SCTP	1064 DATA (TSN=4)
24	0.043514	192.168.0.100	192.168.0.102	SCTP	1064 DATA (TSN=5)
25	0.043654	192.168.0.101	192.168.0.100	SCTP	64 SACK (Ack=5, Arwnd=107568)
26	0.056957	192.168.0.100	192.168.0.102	SCTP	1064 DATA (TSN=6)
27	0.107983	192.168.0.102	192.168.0.100	SCTP	80 ASCONF
28	0.108188	192.168.0.100	192.168.0.102	SCTP	62 ASCONF_ACK
29	0.108299	192.168.0.102	192.168.0.100	SCTP	92 DATA (TSN=5)
30	0.108445	192.168.0.100	192.168.0.102	SCTP	64 SACK (Ack=5, Arwnd=109516)
31	0.108464	192.168.0.102	192.168.0.100	SCTP	92 DATA (TSN=6)
32	0.109175	192.168.0.102	192.168.0.100	SCTP	64 SACK (Ack=6, Arwnd=109568)
33	0.109252	192.168.0.102	192.168.0.100	SCTP	92 DATA (TSN=7)
35	0.109419	192.168.0.100	192.168.0.102	SCTP	64 SACK (Ack=7, Arwnd=109516)

Les paquets ASCONF sont utilisés pour communiquer un changement de configuration et doivent être *acknowledged* par un paquet ASCONF_ACK. Sur l'image précédente, les paquets 6/8, 17/18 et 27/28 sont des échanges ASCONF/ASCONF_ACK.

Il est également possible de voir des paquets HEARTBEAT Chunks et HEARTBEAT ACK Chunks qui sont utilisés pour une vérification périodique de la disponibilité des points de terminaison. Ceux-ci ne sont pas présent dans la capture.

3. SCTP association termination

La terminaison de l'association peut être initiée par n'importe quel point de terminaison avec un paquet contenant un SHUTDOWN chunk. Le point de terminaison destinataire répond avec un paquet contenant un SHUTDOWN ACK chunk, et le point de terminaison initiateur conclut la fermeture avec un paquet contenant un SHUTDOWN COMPLETE chunk. Dans la capture, on remarque que l'initiateur de la fermeture envoie deux paquets SHUTDOWN chunk (paquets 34 et 36) à suivre avant de recevoir un SHUTDOWN ACK chunk (paquet 35). Ici la numérotation des paquets est trompeur car si l'on regarde les *timestamps*, on remarque que le paquet 36 est envoyé avant le paquet 35.

34	0.109345	192.168.0.100	192.168.0.102	SCTP	62 SHUTDOWN
36	0.109508	192.168.0.102	192.168.0.100	SCTP	52 SHUTDOWN_ACK
37	0.109449	192.168.0.100	192.168.0.102	SCTP	62 SHUTDOWN
38	0.109634	192.168.0.100	192.168.0.102	SCTP	62 SHUTDOWN_COMPLETE

Sources :

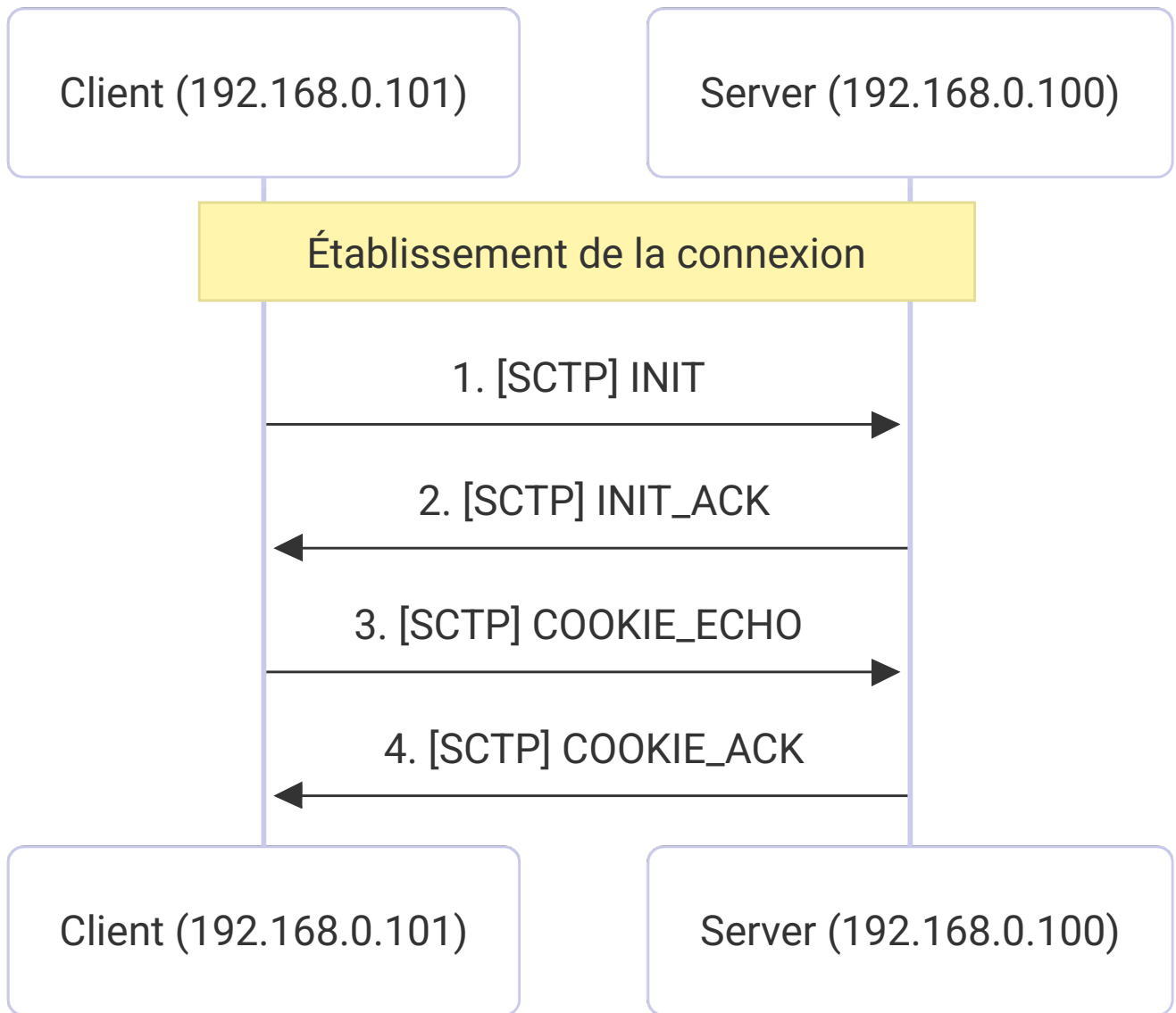
- Oracle Documentation - SCTP Message Flow : https://docs.oracle.com/cd/E80921_01/html/esbc_ecz740_configuration/GUID-E6214D44-39E0-4B00-A491-06A5194CB820.htm
- RFC 5061 - Stream Control Transmission Protocol (SCTP) Dynamic Address Reconfiguration : <https://www.rfc-editor.org/rfc/rfc5061>

Diagramme de séquence

Dessiner un diagramme en flèche (diagramme de séquence) montrant :

- L'émetteur à gauche, le récepteur à droite
- Chaque paquet représenté par une flèche avec :
 - Le numéro du paquet
 - Le ou les types de chunks contenus
 - Une brève description du rôle du paquet
- Les différentes phases clairement identifiées :
 - Phase d'établissement de l'association (4-way handshake)
 - Phase de transfert de données
 - Phase de fermeture (si présente)

Le diagramme de séquence ci-dessous illustre la phase d'établissement de l'association SCTP entre les stations.

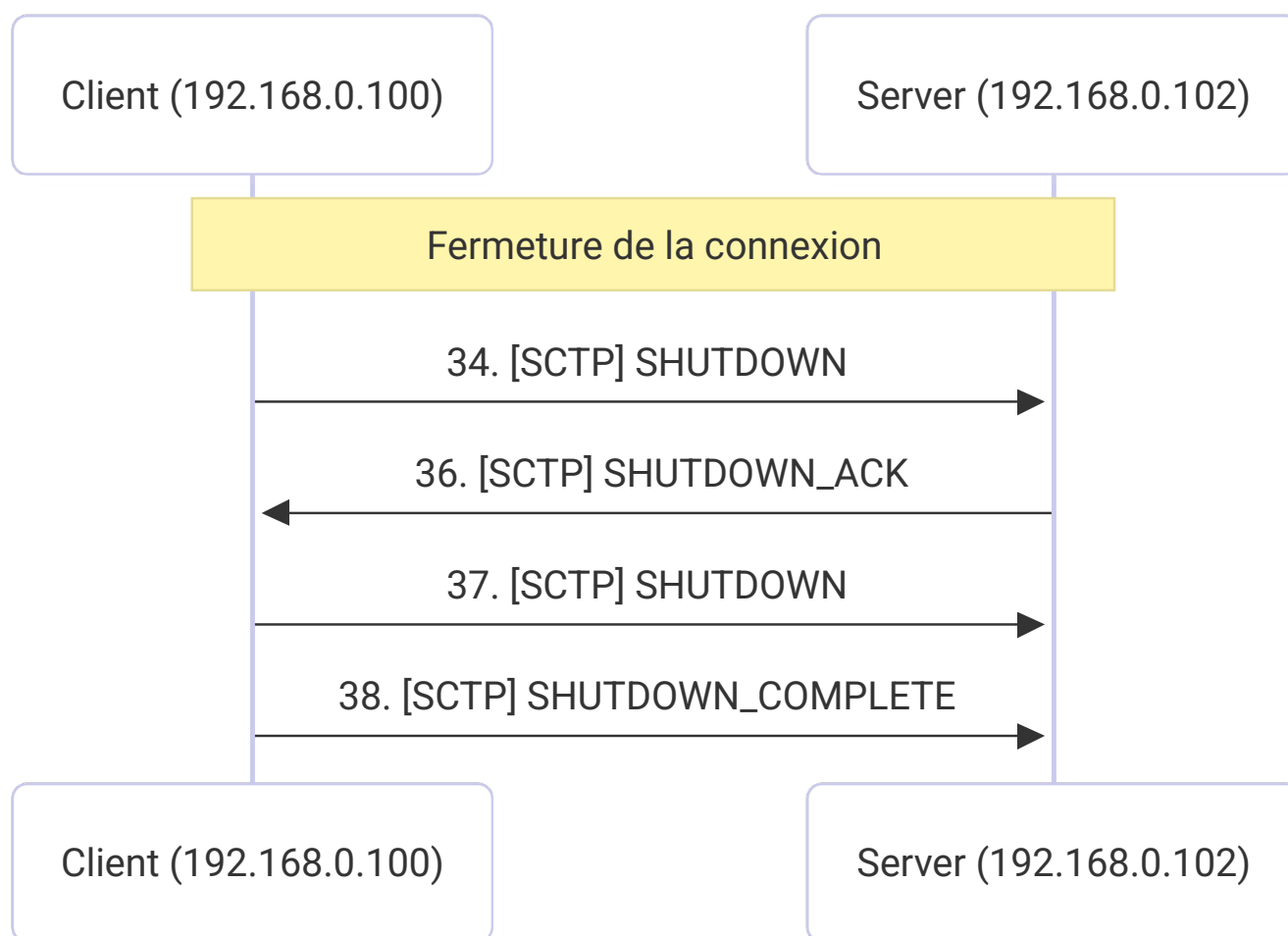


Le diagramme de séquence ci-dessous illustre la phase de transfert de données SCTP entre les stations.

Échange de données et Configuration (Phase 2)



Le diagramme de séquence ci-dessous illustre la phase de fermeture de l'association SCTP entre les stations.



Analyse détaillée des chunks

Pour chaque paquet identifié, indiquer :

1. Le numéro du paquet dans la capture
2. La direction (Client → Serveur ou Serveur → Client)
3. Les chunks présents dans le paquet
4. Pour chaque chunk, expliquer :
 - Son rôle et sa fonction
 - Les paramètres importants qu'il contient
 - Sa place dans le protocole SCTP

1. [Paquet 1] - Client (192.168.0.101) → Serveur (192.168.0.100)

- *Chunk* : INIT (1)
 - Rôle/fonction : utilisé pour initier une association SCTP entre deux points de terminaison
 - Paramètres fixes :
 - Source Port : port source du *sender* de l'INIT paquet

- Destination Port : port destination du *receiver* de l'INIT paquet
- Verification Tag : utilisé par le *receiver* du paquet pour valider l'identité du *sender* de ce paquet. La valeur de ce champ doit être identique à la valeur du champ Initiate Tag reçue
- Checksum : *checksum* du paquet
- Initiate Tag : *tag* de vérification utilisé pour identifier l'association ; présent dans le champ Verification Tag de tous les paquets SCTP du *receiver* de l'INIT paquet.
- Advertised Receiver Window Credit : taille en *bytes* du *buffer* alloué par le *sender* de l'INIT pour cette fenêtre de réception
- Number of Outbound Streams : nombre de flux sortants que le *sender* de l'INIT souhaite créer dans cette association
- Number of Inbound Streams : nombre de flux entrants que le *sender* de l'INIT souhaite créer dans cette association
- Initial TSN : définit le numéro de séquence initial que le *sender* de l'INIT souhaite utiliser pour cette association
- Paramètres variables :
 - IPv4 Address : contient une adresse IPv4 du *sender* de l'INIT
 - IPv6 Address : contient une adresse IPv6 de *sender* de l'INIT
 - Cookie Preservative : utiliser pour suggérer au récepteur de l'INIT une durée de vie plus longue pour le *State Cookie*
 - Reserved for ECN Capable : réservé pour une future utilisation avec *Explicit Congestion Notification (ECN)*
 - Host Name Address : contient le nom d'hôte *DNS* du *sender* de l'INIT
 - Supported Address Types : liste des types d'adresses supportés par l'initiateur
- Remarque.s :
 - le champ Verification Tag contient la valeur 0x00000000 pour les paquets INIT
- Sources :
 - RFC 4960 - Stream Control Transmission Protocol : <https://datatracker.ietf.org/doc/html/rfc4960#section-3.3.2>
 - Wikipedia - SCTP packet structure : https://en.wikipedia.org/wiki/SCTP_packet_structure
 - IBM Documentation - SCTP Common Header Field Descriptions : <https://datatracker.ietf.org/doc/html/rfc4960#section-3.1>

```

▶ Frame 1: Packet, 84 bytes on wire (672 bits), 84 bytes captured (672 bits)
▶ Linux cooked capture v1
▶ Internet Protocol Version 4, Src: 192.168.0.101, Dst: 192.168.0.100
▶ Stream Control Transmission Protocol, Src Port: 6666 (6666), Dst Port: 9999 (9999)
  Source port: 6666
  Destination port: 9999
  Verification tag: 0x00000000
  [Association index: disabled (enable in preferences)]
  Checksum: 0x561ec1fb [unverified]
  [Checksum Status: Unverified]
  ▾ INIT chunk (Outbound streams: 10, inbound streams: 65535)
    ▾ Chunk type: INIT (1)
      0... .. = Bit: Stop processing of the packet
      .0... .. = Bit: Do not report
      Chunk flags: 0x00
      Chunk length: 36
      Initiate tag: 0x71b81d1f
      Advertised receiver window credit (a_rwnd): 109568
      Number of outbound streams: 10
      Number of inbound streams: 65535
      Initial TSN: 2702200202
      ▾ Supported address types parameter (Supported types: IPv4)
        ▾ Parameter type: Supported address types (0x000c)
          0... .. = Bit: Stop processing of chunk
          .0... .. = Bit: Do not report
          Parameter length: 6
          Supported address type: IPv4 address (5)
          Parameter padding: 0000
      ▾ ECN parameter
        ▾ Parameter type: ECN (0x8000)
          1... .. = Bit: Skip parameter and continue processing of the chunk
          .0... .. = Bit: Do not report
          Parameter length: 4
      ▾ Forward TSN supported parameter
        ▾ Parameter type: Forward TSN supported (0xc000)
          1... .. = Bit: Skip parameter and continue processing of the chunk
          .1... .. = Bit: Do report
          Parameter length: 4

```

1. [Paquet 2] - Serveur (192.168.0.100) → Client (192.168.0.101)

- **Chunk : INIT_ACK**
 - Rôle/fonction : utilisé pour *acknowledge* l'initiation d'une association SCTP
 - Paramètres fixes :
 - Source Port : port source du *sender* de l'INIT_ACK paquet
 - Destination Port : port destination du *receiver* de l'INIT_ACK
 - Verification Tag : utilisé par le *receiver* du paquet pour valider l'identité du *sender* de ce paquet. La valeur de ce champ doit être identique à la valeur du champ Initiate Tag de l'INIT paquet
 - Checksum : *checksum* du paquet
 - Initiate Tag : *tag* de vérification utilisé pour identifier l'association ; présent dans le champ Verification Tag de tous les paquets SCTP du *receiver* de l'INIT_ACK paquet.
 - Advertised Receiver Window Credit : taille en *bytes* du *buffer* alloué par le *sender* de l'INIT_ACK pour cette fenêtre de réception
 - Number of Outbound Streams : nombre de flux sortants que le *sender* de l'INIT_ACK souhaite créer dans cette association
 - Number of Inbound Streams : nombre de flux entrants que le *sender* de l'INIT_ACK souhaite créer dans cette association
 - Initial TSN : définit le numéro de séquence initial que le *sender* de l'INIT_ACK souhaite utiliser pour cette association
 - Paramètres variables :

- State Cookie : contient un *Message Authentication Code (MAC)*, un *timestamp* de création du cookie, la durée de vie du cookie et les informations nécessaires pour établir l'association. Le MAC est calculé par le serveur à partir d'une clé secrète connue uniquement de lui.
- IPv4 Address : contient une adresse IPv4 du *sender* de l'INIT_ACK
- IPv6 Address : contient une adresse IPv6 de *sender* de l'INIT_ACK
- Unrecognized Parameter : rapport des paramètres d'INIT non reconnus par le *receiver* de l'INIT paquet
- Reserved for ECN Capable : réservé pour une future utilisation avec *Explicit Congestion Notification (ECN)*
- Host Name Address : contient le nom d'hôte *DNS* du *sender* de l'INIT_ACK
- Remarque.s :
 - le champ Verification Tag du paquet contient la valeur du champ Initiate Tag de l'INIT paquet
- Sources :
 - RFC 4960 - Stream Control Transmission Protocol : <https://datatracker.ietf.org/doc/html/rfc4960#section-3.3.3>
 - Wikipedia - SCTP packet structure : https://en.wikipedia.org/wiki/SCTP_packet_structure
 - IBM Documentation - SCTP association startup and shutdown : <https://www.ibm.com/docs/en/aix/7.2.0?topic=protocol-sctp-association-startup-shutdown>
 - RFC 4960 - Reporting of Unrecognized Parameters : <https://www.rfc-editor.org/rfc/rfc4960#section-3.2.2>

```

> Frame 2: Packet, 240 bytes on wire (1920 bits), 240 bytes captured (1920 bits)
> Linux cooked capture v1
> Internet Protocol Version 4, Src: 192.168.0.100, Dst: 192.168.0.101
> Stream Control Transmission Protocol, Src Port: 9999 (9999), Dst Port: 6666 (6666)
  Source port: 9999
  Destination port: 6666
  Verification tag: 0x71b81dif
  Association index: disabled (enable in preferences)
  Checksum: 0xf68a32a1 [unverified]
  Checksum status: Unverified
  INIT_ACK chunk (Outbound streams: 10, inbound streams: 10)
    Chunk type: INIT ACK (2)
      0... .. = Bit: Stop processing of the packet
      .0... .. = Bit: Do not report
      Chunk flags: 0x00
      Chunk length: 192
      Initiate tag: 0x48e63127
      Advertised receiver window credit (a_rwnd): 109568
      Number of outbound streams: 10
      Number of inbound streams: 10
      Initial TSN: 4194126429
      State cookie parameter (Cookie length: 160 bytes)
        Parameter type: State cookie (0x0007)
          0... .. = Bit: Stop processing of chunk
          .0... .. = Bit: Do not report
          Parameter length: 164
          State cookie [...]: 41f6c2fe873ab164604db8524d8a33d0000000000000000000000000000000002731e6481f1db8710000000000000000c531da411bf603000a00a00
        ECN parameter
          Parameter type: ECN (0x8000)
            1... .. = Bit: Skip parameter and continue processing of the chunk
            .0... .. = Bit: Do not report
            Parameter length: 4
          Forward TSN supported parameter
            Parameter type: Forward TSN supported (0xc000)
              1... .. = Bit: Skip parameter and continue processing of the chunk
              .1... .. = Bit: Do report
              Parameter length: 4

```

1. [Paquet 3] - Client (192.168.0.101) → Serveur (192.168.0.100)

- *Chunk* : COOKIE_ECHO
 - Rôle/fonction : utilisé pour répondre à un INIT_ACK paquet. Ce *chunk* ne fait que renvoyer le *State Cookie* reçu dans le INIT_ACK paquet
 - Paramètres fixes :
 - Source Port : port source du *sender* de l'COOKIE_ECHO paquet
 - Destination Port : port destination du *receiver* de l'COOKIE_ECHO
 - Verification Tag : utilisé par le *receiver* du paquet pour valider l'identité du *sender* de ce paquet. La valeur de ce champ doit être identique à la valeur du champ Initiate Tag de l'INIT_ACK paquet

- Checksum : *checksum* du paquet
- Cookie : contient le *State Cookie* reçu dans le INIT_ACK paquet
- Paramètres variables : aucun
- Remarque.s :
 - le champ Verification Tag du paquet contient la valeur du champ Initiate Tag de l'INIT_ACK paquet
 - le champ Cookie contient la même valeur que le champ State Cookie du INIT_ACK paquet
- Sources :
 - RFC 4960 - Stream Control Transmission Protocol : <https://datatracker.ietf.org/doc/html/rfc4960#section-3.3.11>
 - IBM Documentation - SCTP association startup and shutdown : <https://www.ibm.com/docs/en/aix/7.2.0?topic=protocol-sctp-association-startup-shutdown>
 - Wikipedia - SCTP packet structure : https://en.wikipedia.org/wiki/SCTP_packet_structure

[illegible]

1. [Paquet 4] - Serveur (192.168.0.100) → Client (192.168.0.101)

- **Chunk** : COOKIE_ACK
 - Rôle/fonction : utilisé pour *acknowledge* la réception d'un COOKIE_ECHO paquet et ainsi compléter l'établissement de l'association SCTP
 - Paramètres fixes :
 - Source Port : port source du *sender* de l'COOKIE_ACK paquet
 - Destination Port : port destination du *receiver* de l'COOKIE_ACK
 - Verification Tag : utilisé par le *receiver* du paquet pour valider l'identité du *sender* de ce paquet. La valeur de ce champ doit être identique à la valeur du champ Initiate Tag de l'INIT_ACK paquet
 - Checksum : *checksum* du paquet
 - Paramètres variables : aucun
 - Remarques :
 - le champ Verification Tag du paquet contient la valeur du champ Initiate Tag de l'INIT_ACK paquet
 - Sources :
 - RFC 4960 - Stream Control Transmission Protocol : <https://datatracker.ietf.org/doc/html/rfc4960#section-3.3.12>
 - IBM Documentation - SCTP association startup and shutdown : <https://www.ibm.com/docs/en/aix/7.2.0?topic=protocol-sctp-association-startup-shutdown>
 - Wikipedia - SCTP packet structure : https://en.wikipedia.org/wiki/SCTP_packet_structure

```

▶ Frame 4: Packet, 62 bytes on wire (496 bits), 62 bytes captured (496 bits)
▶ Linux cooked capture v1
▶ Internet Protocol Version 4, Src: 192.168.0.100, Dst: 192.168.0.101
▼ Stream Control Transmission Protocol, Src Port: 9999 (9999), Dst Port: 6666 (6666)
  Source port: 9999
  Destination port: 6666
  Verification tag: 0x71b81d1f
  [Association index: disabled (enable in preferences)]
  Checksum: 0x4fb914a0 [unverified]
  [Checksum Status: Unverified]
  ▼ COOKIE_ACK chunk
    ▼ Chunk type: COOKIE_ACK (11)
      0... .. = Bit: Stop processing of the packet
      .0... .. = Bit: Do not report
      Chunk flags: 0x00
      Chunk length: 4

```

1. [Paquet 5] - Client (192.168.0.101) → Serveur (192.168.0.100)

• Chunk : DATA

- Rôle/fonction : utilisé pour transférer des données entre les points de terminaison d'une association SCTP
- Paramètres fixes :
 - Source Port : port source du *sender* du paquet DATA
 - Destination Port : port destination du *receiver* du paquet DATA
 - Verification Tag : utilisé par le *receiver* du paquet pour valider l'identité du *sender* du paquet DATA
 - Checksum : *checksum* du paquet
 - I-Bit : (*I*)*mmmediate bit* : si égal à 1, indique que le *sender* du paquet DATA souhaite que le *receiver* du paquet lui envoie un SACK immédiatement après la réception de ce paquet DATA
 - U-Bit : (*U*)*nordered bit* : si égal à 1, indique que c'est un paquet de données non ordonné et donc qu'il n'y a pas de numéro de Stream Sequence Number assigné à ce paquet (le *receiver* doit ignorer le champ Stream Sequence Number dans ce cas)
 - B-Bit : (*B*)*eginning fragment bit* : si égal à 1, indique que c'est le premier fragment d'un message utilisateur
 - E-Bit : (*E*)*nding fragment bit* : si égal à 1, indique que c'est le dernier fragment d'un message utilisateur
 - Transmission Sequence Number (relative) (TSN) : numéro de séquence de transmission relatif à ce paquet DATA dans l'association SCTP
 - Transmission Sequence Number (absolute) (TSN) : numéro de séquence de transmission absolu à ce paquet DATA dans l'association SCTP
 - Stream Identifier (SID) : identifiant du flux auquel ce paquet DATA appartient
 - Stream Sequence Number (SSN) : numéro de séquence du paquet DATA dans le flux auquel il appartient
 - Payload Protocol Identifier : spécifie un protocole de couche supérieure (application) pour ce paquet DATA
 - User Data : données de l'application transportées par ce paquet DATA
- Paramètres variables : aucun
- Remarque.s :
 - Le champ Verification Tag du paquet contient la valeur du champ Initiate Tag de l'INIT_ACK paquet
 - Le champ TSN est à 0 (premier paquet DATA de l'association)

- Le champ SID est à 0 (premier flux de l'association)
- Le champ SSN est à 0 (premier paquet DATA du flux)
- Le champ Payload Protocol Identifier est à 0 (pas de protocole de couche supérieure spécifié)
- Sources :
 - RFC 4960 - Stream Control Transmission Protocol : <https://datatracker.ietf.org/doc/html/rfc4960#section-3.3.1>
 - Wikipedia - SCTP packet structure : https://en.wikipedia.org/wiki/SCTP_packet_structure
 - RFC 7053 - SACK-IMMEDIATELY Extension for the Stream Control Transmission Protocol : <https://www.rfc-editor.org/rfc/rfc7053>

```

Frame 5: Packet, 92 bytes on wire (736 bits), 92 bytes captured (736 bits)
Linux cooked capture v1
Internet Protocol Version 4, Src: 192.168.0.101, Dst: 192.168.0.100
Stream Control Transmission Protocol, Src Port: 6666 (6666), Dst Port: 9999 (9999)
  Source port: 6666
  Destination port: 9999
  Verification tag: 0x48e63127
  [Association index: disabled (enable in preferences)]
  Checksum: 0x1adf8d92 [unverified]
  [Checksum status: unverifiable]
  DATA chunk (ordered, complete segment, TSN: 0, SID: 0, SSN: 0, PPID: 0, payload length: 26 bytes)
  Data (26 bytes)
    Data: 5468616e6b20796f752021204d722e205365727665722021200a
    [Length: 26]

```

1. [Paquet 6] - Client (192.168.0.101) → Serveur (192.168.0.100)

- *Chunk* : ASCONF
 - Rôle/fonction : utilisé pour communiquer une demande de changement de configuration qui doit être *acknowledge*.
 - Paramètres fixes :
 - Source Port : port source du *sender* du paquet ASCONF
 - Destination Port : port destination du *receiver* du paquet ASCONF
 - Verification Tag : utilisé par le *receiver* du paquet pour valider l'identité du *sender* du paquet ASCONF
 - Checksum : *checksum* du paquet
 - Sequence Number : numéro de séquence permettant d'identifier de manière unique chaque paquet ASCONF dans une association SCTP
 - Address Parameter : indique l'adresse IP concernée par la demande de changement de configuration
 - ASCONF Parameter : indique le type de changement de configuration demandé (ajout ou suppression d'une adresse IP, changement de l'adresse IP principale)
 - Paramètres variables : aucun
 - Remarques :
 - Le champ Verification Tag du paquet contient la valeur du champ Initiate Tag de l'INIT_ACK paquet
 - Le champ Sequence Number du paquet est à 0xa1104d8a (premier paquet ASCONF de l'association)
 - Le champ Address Parameter du paquet contient l'adresse IP 192.168.0.101
 - Le champ ASCONF Parameter est nommé "*Add IP address parameter*" et indique que le changement de configuration demandé est l'ajout de l'adresse IPv4 192.168.0.102
- Sources :

- RFC 5061 - Stream Control Transmission Protocol (SCTP) Dynamic Address Reconfiguration : <https://www.rfc-editor.org/rfc/rfc5061#section-4.1.1>
- Wikipedia - SCTP packet structure : https://en.wikipedia.org/wiki/SCTP_packet_structure

```

▶ Frame 6: Packet, 80 bytes on wire (640 bits), 80 bytes captured (640 bits)
▶ Linux cooked capture v1
▶ Internet Protocol Version 4, Src: 192.168.0.101, Dst: 192.168.0.100
▼ Stream Control Transmission Protocol, Src Port: 6666 (6666), Dst Port: 9999 (9999)
  Source port: 6666
  Destination port: 9999
  Verification tag: 0x48e63127
  [Association index: disabled (enable in preferences)]
  Checksum: 0x500d88fe [unverified]
  [Checksum status: unverified]
  ▼ ASCONF chunk
    ▶ Chunk type: ASCONF (193)
    Chunk flags: 0x00
    Chunk length: 32
    Sequence number: 0xa1104d8a
    ▼ IPv4 address parameter (Address: 192.168.0.101)
      ▶ Parameter type: IPv4 address (0x0005)
      Parameter length: 8
      IP Version 4 address: 192.168.0.101
    ▼ Add IP address parameter (Address: 192.168.0.102, correlation ID: 0)
      ▶ Parameter type: Add IP address (0xc001)
      1... .. = Bit: Skip parameter and continue processing of the chunk
      .1... .. = Bit: Do report
      Parameter length: 16
      Correlation_id: 0x00000000
    ▼ IPv4 address parameter (Address: 192.168.0.102)
      ▶ Parameter type: IPv4 address (0x0005)
      Parameter length: 8
      IP Version 4 address: 192.168.0.102

```

1. [Paquet 7] - Serveur (192.168.0.100) → Client (192.168.0.101)

- *Chunk* : SACK
 - Rôle/fonction : utilisé pour *acknowledge* la réception de paquets DATA et informer le *sender* de l'écart dans les séquences des paquets DATA reçus (TSN)
 - Paramètres fixes :
 - Source Port : port source du *sender* du paquet SACK
 - Destination Port : port destination du *receiver* du paquet SACK
 - Verification Tag : utilisé par le *receiver* du paquet pour valider l'identité du *sender* du paquet SACK
 - Checksum : *checksum* du paquet
 - Cumulative TSN Ack (relative) : contient le numéro de séquence de transmission relatif du dernier paquet DATA reçu en séquence avant écart
 - Cumulative TSN Ack (absolute) : contient le numéro de séquence de transmission absolu du dernier paquet DATA reçu en séquence avant écart
 - Advertised Receiver Window Credit (a_rwnd) : mise à jour de la taille en *bytes* du *buffer* alloué par le *sender* du paquet SACK pour la fenêtre de réception
 - Number of Gap Ack Blocks : nombre de blocs d'*acknowledgment* d'écart dans les séquences des paquets DATA reçus
 - Number of Duplicate TSNs : indique le nombre de numéros de séquence de transmission de paquets DATA reçus plus d'une fois
 - Paramètres variables : aucun
 - Remarque.s :
 - Le champ Verification Tag du paquet contient la valeur du champ Initiate Tag de l'INIT paquet

- Le champ Cumulative TSN Ack (relative) du paquet est à 0 (le dernier paquet DATA reçu ; paquet 5)
- Le champ Number of Gap Ack Blocks du paquet est à 0 (pas d'écart dans les séquences des paquets DATA reçus)
- Le champ Number of Duplicate TSNs du paquet est à 0 (pas de numéros de séquence de transmission de paquets DATA reçus plus d'une fois)
- Sources :
 - RFC 4960 - Stream Control Transmission Protocol : <https://datatracker.ietf.org/doc/html/rfc4960#section-3.3.4>
 - Wikipedia - SCTP packet structure : https://en.wikipedia.org/wiki/SCTP_packet_structure

```

▶ Frame 6: Packet, 80 bytes on wire (640 bits), 80 bytes captured (640 bits)
▶ Linux cooked capture v1
▶ Internet Protocol Version 4, Src: 192.168.0.101, Dst: 192.168.0.100
▼ Stream Control Transmission Protocol, Src Port: 6666 (6666), Dst Port: 9999 (9999)
  Source port: 6666
  Destination port: 9999
  Verification tag: 0x48e63127
  [Association index: disabled (enable in preferences)]
  Checksum: 0x500d88fe [unverified]
  [Unverified checksum: 0x500d88fe]
  ▼ ASCONF chunk
    ▶ Chunk type: ASCONF (193)
    Chunk flags: 0x00
    Chunk length: 32
    Sequence number: 0xa1104d8a
    ▼ IPv4 address parameter (Address: 192.168.0.101)
      Parameter type: IPv4 address (0x0005)
      Parameter length: 8
      IP Version 4 address: 192.168.0.101
    ▼ Add IP address parameter (Address: 192.168.0.102, correlation ID: 0)
      Parameter type: Add IP address (0xc001)
      1... .. = Bit: Skip parameter and continue processing of the chunk
      .1... .. = Bit: Do report
      Parameter length: 16
      Correlation_id: 0x00000000
      ▼ IPv4 address parameter (Address: 192.168.0.102)
        Parameter type: IPv4 address (0x0005)
        Parameter length: 8
        IP Version 4 address: 192.168.0.102

```

1. [Paquet 8] - Serveur (192.168.0.100) → Client (192.168.0.101)

- Chunk : ASCONF_ACK
 - Rôle/fonction : TODO
 - Paramètres fixes : TODO
 - Paramètres variables : TODO
 - Sources :
 - Wikipedia - SCTP packet structure : https://en.wikipedia.org/wiki/SCTP_packet_structure

2. [Paquet 9] - Serveur (192.168.0.100) → Client (192.168.0.101)

- Chunk : DATA, voir paquet 5 pour les détails

3. [Paquet 10] - Client (192.168.0.101) → Serveur (192.168.0.100)

- Chunk : SACK, voir paquet 7 pour les détails

4. [Paquet 11] - Client (192.168.0.101) → Serveur (192.168.0.100)

- Chunk : DATA, voir paquet 5 pour les détails

5. [Paquet 12] - Serveur (192.168.0.100) → Client (192.168.0.101)

- Chunk : SACK, voir paquet 7 pour les détails
- Chunk : DATA, voir paquet 5 pour les détails

6. **[Paquet 13] - Serveur (192.168.0.100) → Client (192.168.0.101)**
 - *Chunk* : DATA, voir paquet 5 pour les détails
7. **[Paquet 14] - Client (192.168.0.101) → Serveur (192.168.0.100)**
 - *Chunk* : SACK, voir paquet 7 pour les détails
8. **[Paquet 15] - Serveur (192.168.0.100) → Client (192.168.0.101)**
 - *Chunk* : DATA, voir paquet 5 pour les détails
9. **[Paquet 16] - Client (192.168.0.101) → Serveur (192.168.0.100)**
 - *Chunk* : DATA, voir paquet 5 pour les détails
10. **[Paquet 17] - Client (192.168.0.101) → Serveur (192.168.0.100)**
 - *Chunk* : ASCONF, voir paquet 6 pour les détails
11. **[Paquet 18] - Serveur (192.168.0.100) → Serveur (192.168.0.102)**
 - *Chunk* : ASCONF_ACK, voir paquet 8 pour les détails
12. **[Paquet 19] - Client (192.168.0.101) → Serveur (192.168.0.100)**
 - *Chunk* : DATA, voir paquet 5 pour les détails
13. **[Paquet 20] - Serveur (192.168.0.100) → Client (192.168.0.101)**
 - *Chunk* : SACK, voir paquet 7 pour les détails
14. **[Paquet 21] - Client (192.168.0.101) → Serveur (192.168.0.100)**
 - *Chunk* : SACK, voir paquet 7 pour les détails
15. **[Paquet 22] - Client (192.168.0.101) → Serveur (192.168.0.100)**
 - *Chunk* : DATA, voir paquet 5 pour les détails
16. **[Paquet 23] - Serveur (192.168.0.100) → Serveur (192.168.0.102)**
 - *Chunk* : DATA, voir paquet 5 pour les détails
17. **[Paquet 24] - Serveur (192.168.0.100) → Serveur (192.168.0.102)**
 - *Chunk* : DATA, voir paquet 5 pour les détails
18. **[Paquet 25] - Client (192.168.0.101) → Serveur (192.168.0.100)**
 - *Chunk* : SACK, voir paquet 7 pour les détails
19. **[Paquet 26] - Serveur (192.168.0.100) → Serveur (192.168.0.102)**
 - *Chunk* : DATA, voir paquet 5 pour les détails
20. **[Paquet 27] - Serveur (192.168.0.102) → Serveur (192.168.0.100)**
 - *Chunk* : ASCONF, voir paquet 6 pour les détails
21. **[Paquet 28] - Serveur (192.168.0.100) → Serveur (192.168.0.102)**
 - *Chunk* : ASCONF_ACK, voir paquet 8 pour les détails
22. **[Paquet 29] - Serveur (192.168.0.102) → Serveur (192.168.0.100)**
 - *Chunk* : DATA, voir paquet 5 pour les détails
23. **[Paquet 30] - Serveur (192.168.0.100) → Serveur (192.168.0.102)**
 - *Chunk* : SACK, voir paquet 7 pour les détails
24. **[Paquet 31] - Serveur (192.168.0.102) → Serveur (192.168.0.100)**
 - *Chunk* : DATA, voir paquet 5 pour les détails
25. **[Paquet 32] - Serveur (192.168.0.102) → Serveur (192.168.0.100)**
 - *Chunk* : SACK, voir paquet 7 pour les détails
26. **[Paquet 33] - Serveur (192.168.0.102) → Serveur (192.168.0.100)**
 - *Chunk* : DATA, voir paquet 5 pour les détails
27. **[Paquet 34] - Serveur (192.168.0.100) → Client (192.168.0.102)**
 - *Chunk* : SHUTDOWN

- Rôle/fonction : TODO
- Paramètres fixes : TODO
- Paramètres variables : TODO
- Sources :

- Wikipedia - SCTP packet structure : https://en.wikipedia.org/wiki/SCTP_packet_structure

28. **[Paquet 35] - Serveur (192.168.0.100) → Serveur (192.168.0.102)**

- *Chunk* : SACK, voir paquet 7 pour les détails

29. **[Paquet 36] - Client (192.168.0.102) → Serveur (192.168.0.100)**

- *Chunk* : SHUTDOWN_ACK
 - Rôle/fonction : TODO
 - Paramètres fixes : TODO
 - Paramètres variables : TODO
 - Sources :

- Wikipedia - SCTP packet structure : https://en.wikipedia.org/wiki/SCTP_packet_structure

30. **[Paquet 37] - Serveur (192.168.0.100) → Client (192.168.0.102)**

- *Chunk* : SHUTDOWN
 - Rôle/fonction : voir paquet 34 pour les détails

31. **[Paquet 38] - Serveur (192.168.0.100) → Client (192.168.0.102)**

- *Chunk* : SHUTDOWN_COMPLETE
 - Rôle/fonction : TODO
 - Paramètres fixes : TODO
 - Paramètres variables : TODO
 - Sources :

- Wikipedia - SCTP packet structure : https://en.wikipedia.org/wiki/SCTP_packet_structure

Questions

1. Quelle est la différence principale entre le handshake SCTP (4-way) et le handshake TCP (3-way) ?

Outre le fait que SCTP comporte une étape supplémentaire, la différence majeure réside dans le moment où le serveur alloue ses ressources. Avec TCP, cette allocation se fait dès la réception du SYN. En revanche, avec SCTP, le serveur n'alloue ses ressources qu'à la réception du COOKIE_ECHO, une fois que le client a prouvé sa légitimité.

2. Pourquoi SCTP utilise-t-il un cookie lors de l'établissement de connexion ?

SCTP utilise un cookie pour pallier les vulnérabilités de TCP face aux attaques par déni de service, comme le SYN Flood, qui visent à saturer et faire planter le serveur. Ce cookie agit comme un mécanisme de sécurité : le serveur n'alloue sa mémoire qu'après avoir reçu un COOKIE_ECHO valide, évitant ainsi de gaspiller des ressources pour des requêtes illégitimes.

3. Quel est l'avantage du SACK par rapport à un ACK classique ?

Le principal avantage du SACK est l'optimisation de la bande passante grâce à l'élimination des retransmissions inutiles. Avec un ACK, la perte d'un paquet entraîne souvent la retransmission de celui-ci ainsi que de tous les paquets l'ayant suivi. Le SACK, en revanche, permet d'indiquer précisément les paquets manquants et ceux bien reçus, limitant ainsi le renvoi aux seules données perdues.

4. Dans quel contexte l'extension ADD-IP (ASCONF) est-elle utile ?

L'extension ADD-IP est particulièrement utile pour gérer le multihoming et la mobilité, car elle permet de modifier dynamiquement les adresses IP d'une association en cours sans l'interrompre. Concrètement, si l'interface principale d'un serveur tombe en panne, le chunk ASCONF permet de basculer sur une autre adresse IP. De même, si un téléphone passe d'une connexion Wi-Fi à un réseau 4G/5G, cette extension assurera la continuité de la communication.

Conclusion

TODO